

CMPU 3010 Databases 2

Lab 1: CommunityCare Database

PostgreSQL setup, database investigation and SQL review Solution

Run first:

```
SET search_path TO communitycare;
```

Part A — Investigate the Database

A1. Primary keys

Table	Primary key
Patient	patient_id
Centre	centre_id
Staff	staff_id
Service	service_id
staff_service	(staff_id, service_id)
appointment	appointment_id
Payment	payment_id

A2. Foreign-key relationships

- staff.centre_id → centre.centre_id
- staff_service.staff_id → staff.staff_id
- staff_service.service_id → service.service_id
- appointment.patient_id → patient.patient_id
- appointment.staff_id → staff.staff_id
- appointment.service_id → service.service_id
- payment.appointment_id → appointment.appointment_id

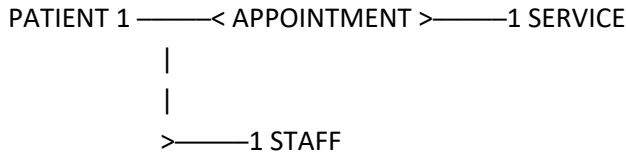
Therefore, conceptually:

CENTRE 1 ———< STAFF

STAFF >————< SERVICE

via

STAFF_SERVICE



APPOINTMENT 1 — 0..1 PAYMENT

A3. staff_service

- staff_service resolves the **many-to-many relationship** between staff and service.
- A staff member may provide several services, while a service may be provided by several staff members.
- Its primary key is:(staff_id, service_id. It is the **combination** that uniquely identifies an assignment.
- For example:

staff_id service_id

1 1

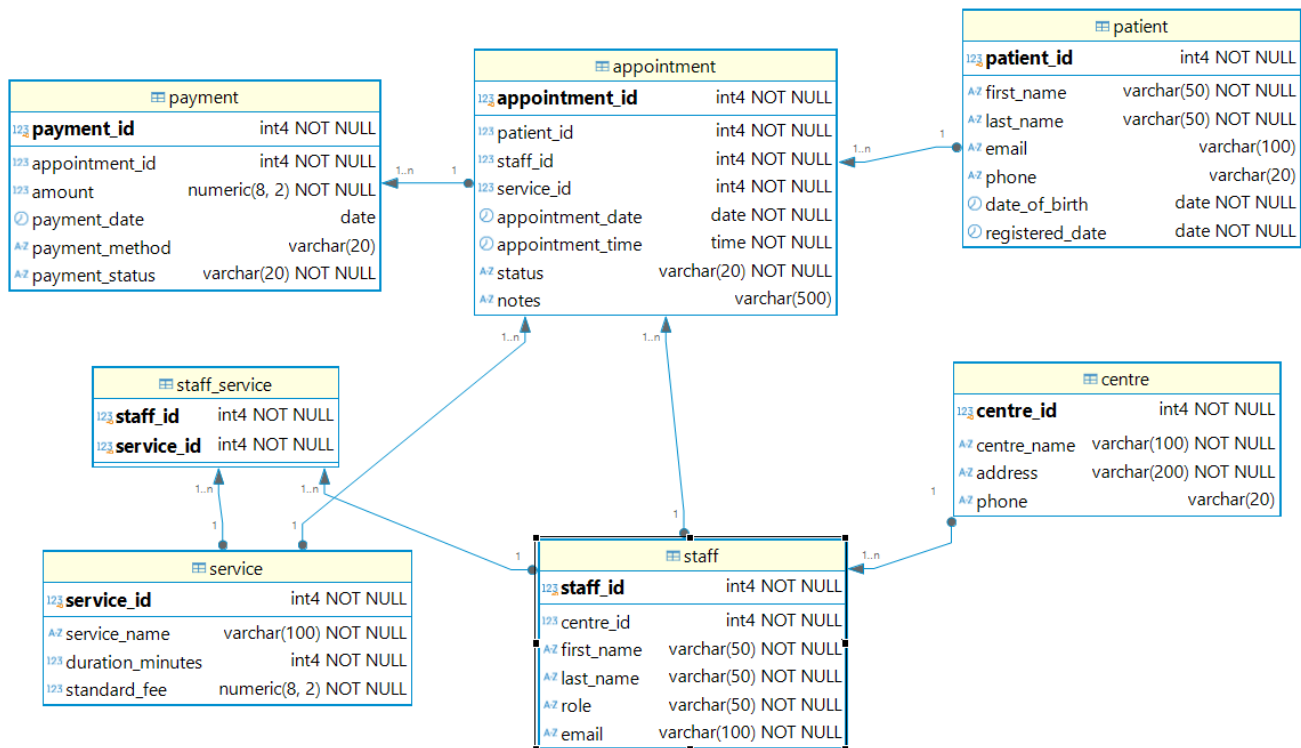
1 3

2 1

The composite PK also prevents the same staff/service combination being inserted twice.

A4. ER diagram

This was generated using DBeaver. All keys, datatypes and nullability is shown – Bachman notation chosen.



A5. Integrity constraints

Examples include:

- primary keys prevent duplicate entity identifiers;
- foreign keys prevent appointments referring to patients, staff or services that do not exist;
- patient.email is unique;
- staff.email is unique;
- centre_name is unique;
- service_name is unique;
- service duration must be greater than zero;
- service fee cannot be negative;
- appointment status is restricted to permitted values;
- a staff member cannot have two appointments beginning at exactly the same date and time;
- payment amount cannot be negative;
- one appointment can have at most one payment record.

A6. Double booking

- No — the same staff member cannot have two appointments **starting at exactly the same date and time**. The appointment table contains a UNIQUE constraint on: `staff_id, appointment_date, appointment_time`
- Therefore PostgreSQL will reject a second appointment with the same combination.
- This doesn't necessarily solve **all** scheduling conflicts. For example, if appointments have different durations, an appointment beginning at 10:00 and another beginning at 10:15 might still overlap. The current constraint only prevents identical start times.

A7. Optional — NULLs

Nullable columns include:

- centre.phone
- patient.email
- patient.phone
- appointment.notes
- payment.payment_date
- payment.payment_method

A8. Optional — Appointment status

Permitted values are:

- Scheduled
- Completed
- Cancelled
- No Show

They are enforced by a CHECK constraint.

Part B — SQL Review

B1. Patients ordered by surname and first name

```
SELECT patient_id,  
       first_name,  
       last_name  
FROM patient  
ORDER BY last_name, first_name;
```

B2. Appointment, patient and service

```
SELECT  
    a.appointment_id,  
    p.first_name,  
    p.last_name,  
    s.service_name,  
    a.appointment_date,  
    a.appointment_time,  
    a.status  
FROM appointment a  
JOIN patient p  
    ON a.patient_id = p.patient_id  
JOIN service s  
    ON a.service_id = s.service_id  
ORDER BY a.appointment_date, a.appointment_time;
```

B3. Cancelled appointments

```
SELECT  
    p.first_name AS patient_first_name,  
    p.last_name AS patient_last_name,  
    st.first_name AS staff_first_name,  
    st.last_name AS staff_last_name,  
    s.service_name,  
    a.appointment_date  
FROM appointment a  
JOIN patient p  
    ON a.patient_id = p.patient_id  
JOIN staff st  
    ON a.staff_id = st.staff_id  
JOIN service s  
    ON a.service_id = s.service_id  
WHERE a.status = 'Cancelled'  
ORDER BY a.appointment_date;
```

B4. Number of appointments for each service

This version has the advantage of making the grouping key explicit:

```
SELECT
    s.service_name,
    COUNT(a.appointment_id) AS number_of_appointments
FROM service s
JOIN appointment a
    ON s.service_id = a.service_id
GROUP BY s.service_id, s.service_name
ORDER BY number_of_appointments DESC;
```

An alternative valid answer is:

```
SELECT
    s.service_name,
    COUNT(*) AS number_of_appointments
FROM appointment a
JOIN service s
    ON a.service_id = s.service_id
GROUP BY s.service_name
ORDER BY number_of_appointments DESC;
```

B5. Services with more than five appointments

The important concept is: WHERE → filters rows HAVING → filters groups

```
SELECT
    s.service_name,
    COUNT(a.appointment_id) AS number_of_appointments
FROM service s
JOIN appointment a
    ON s.service_id = a.service_id
GROUP BY s.service_id, s.service_name
HAVING COUNT(a.appointment_id) > 5
ORDER BY number_of_appointments DESC;
```

B6. Patients who have never had an appointment

```
SELECT
    p.patient_id,
    p.first_name,
    p.last_name
FROM patient p
LEFT JOIN appointment a
    ON p.patient_id = a.patient_id
WHERE a.appointment_id IS NULL;
```

A perfectly valid alternative is:

```
SELECT
    p.patient_id,
    p.first_name,
    p.last_name
FROM patient p
WHERE NOT EXISTS (
    SELECT 1
    FROM appointment a
    WHERE a.patient_id = p.patient_id
);
```

B7. Appointments with no payment

Payment must be LEFT JOINed rather than INNER JOINed:

```
SELECT
    a.appointment_id,
    p.first_name,
    p.last_name,
    a.appointment_date,
    s.service_name
FROM appointment a
JOIN patient p
    ON a.patient_id = p.patient_id
JOIN service s
    ON a.service_id = s.service_id
LEFT JOIN payment pay
    ON a.appointment_id = pay.appointment_id
WHERE pay.payment_id IS NULL
ORDER BY a.appointment_date;
```

Part C — Harder SQL Problem

For each patient, show each appointment and the date of that patient's **previous appointment**.

```
SELECT
    p.patient_id,
    p.first_name,
    p.last_name,
    a.appointment_date,
    (
        SELECT MAX(a2.appointment_date)
        FROM appointment a2
        WHERE a2.patient_id = a.patient_id
          AND a2.appointment_date < a.appointment_date
    ) AS previous_appointment
FROM appointment a
JOIN patient p
  ON p.patient_id = a.patient_id
ORDER BY
    p.patient_id,
    a.appointment_date;
```

This awkward. A cleaner solution could use :

LAG() and:

```
OVER (
    PARTITION BY ...
    ORDER BY ...
)
```

We will be looking at this in our Advanced SQL commands.

Part D — Transaction Teaser

Consider two receptionists trying to book the same final appointment slot at the same time.

Receptionist A checks that 10:00 is available. Before A completes the booking, Receptionist B checks the same slot and also sees it as available. Both then attempt to book it.

D1. What could go wrong?

- Both receptionists could initially see the 10:00 slot as available and both attempt to book it.
- This is a **concurrency/race condition**.
- In the supplied database, the UNIQUE constraint should ultimately prevent two appointments for the same staff member at exactly the same date/time. One transaction may therefore fail when it attempts the conflicting insert.
- However, the application still needs to deal correctly with this situation.

D2. Is BEGIN / COMMIT enough?

- No.
- Simply writing:

```
BEGIN;  
-- check availability  
-- insert booking  
COMMIT;
```

- does not by itself prevent another concurrent transaction from checking the same data between the check and the insert.
- The eventual solution may involve appropriate constraints plus transaction isolation/locking and correct error handling.

D3. What should the database guarantee?

Some possible answers:

- data should remain consistent;
- two users should not be allowed to create an invalid double booking;
- one user's incomplete operation should not leave the database in an invalid state;
- concurrent transactions should be handled safely.

Part E — New Requirement

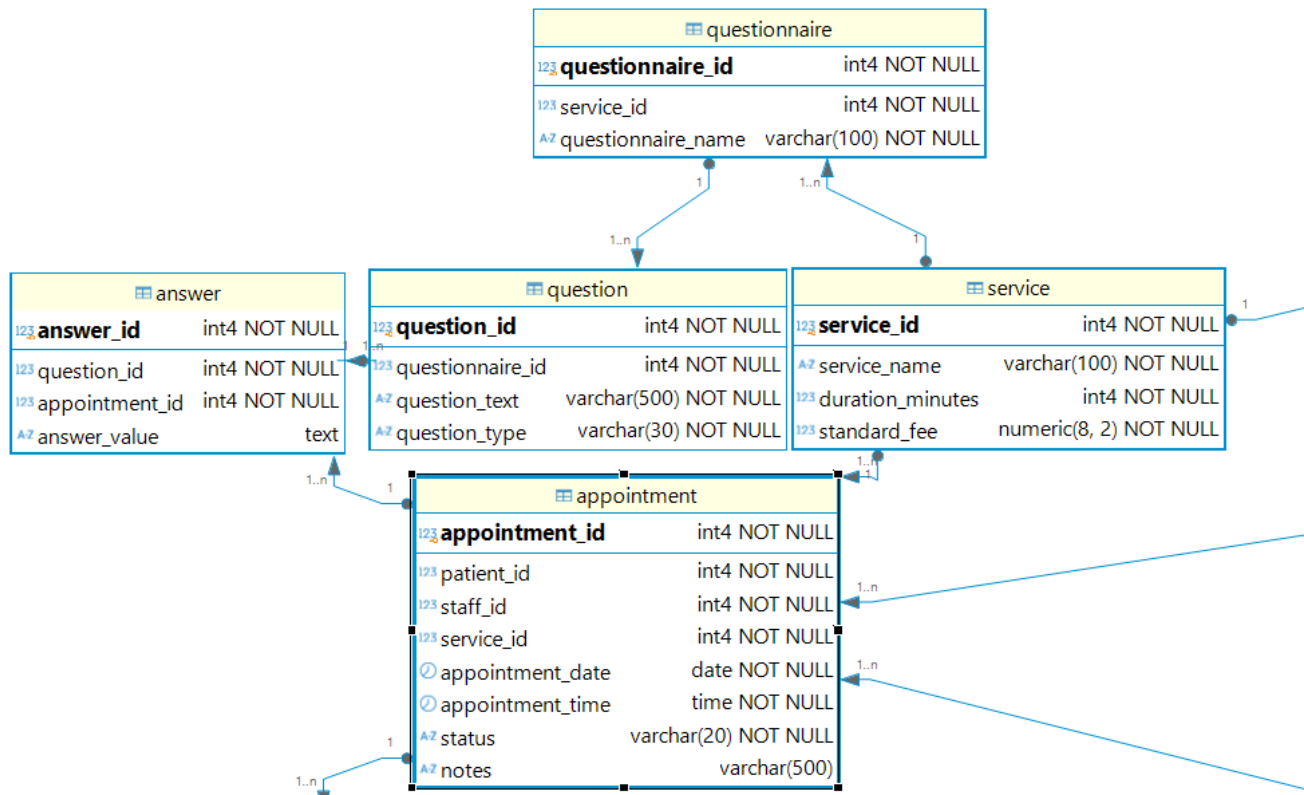
E1. How could questionnaires be stored relationally?

There are multiple possible answers here One possible relational design would introduce tables such as:

```
QUESTIONNAIRE  
-----  
questionnaire_id PK  
service_id FK  
questionnaire_name
```

```
QUESTION  
-----  
question_id PK  
questionnaire_id FK  
question_text  
question_type
```

```
RESPONSE  
-----  
response_id PK  
appointment_id FK  
question_id FK  
response_value
```



For questions allowing multiple answers, such as several pain areas, the model would probably need further tables rather than storing several values in one relational attribute.

E2. What difficulties arise?

Possible difficulties:

- different services require different questions;
- different questions require different data types;
- questionnaires may change over time;
- some questions allow one answer while others allow multiple answers;
- many tables/rows may be required to represent one completed questionnaire;
- retrieving a complete questionnaire could require several joins;
- the structure may become cumbersome as new questionnaire types are introduced.

The important conclusion isn't: "PostgreSQL can't store this."

It certainly can. You should recognise that:

A normalized relational representation is possible, but highly variable questionnaire data may be awkward to model and retrieve.